Object Oriented Programming (CT-260)

Lab 05

main.cpp

```cpp
//Example 1
//Single Inheritance
#include <iostream>
using namespace std;
class base {
public:
    int x;

    void getdata() {
        cout << "Enter the value of x = ";
        cin >> x;
    }
};
class derive : public base {
private:
    int y;
public:
    void readdata() {
        cout << "Enter the value of y = ";
        cin >> y;
    }
    void product() {
        cout << "Product = " << x * y;
    }
};
int main() {
    derive a;
    a.getdata();
    a.readdata();
    a.product();
    return 0;
}
```

Output

```
Enter the value of x = 3
Enter the value of y = 4
Product = 12

=== Code Execution Successful ===
```

main.cpp

```cpp
//Example 2
//Multilevel Inheritance
#include <iostream>
using namespace std;
class base {
public:
    int x;

    void getdata() {
        cout << "Enter value of x= ";
        cin >> x;
    }
};
class derive1 : public base {
public:
    int y;

    void readdata() {
        cout << "\nEnter value of y= ";
        cin >> y;
    }
};
class derive2 : public derive1 {
private:
    int z;

public:
    void indata() {
        cout << "\nEnter value of z= ";
        cin >> z;
    }

    void product() {
        cout << "\nProduct= " << x * y * z;
    }
};
int main() {
    derive2 a;
    a.getdata();
    a.readdata();
    a.indata();
    a.product();
    return 0;
}
```

Output

```
Enter value of x= 2

Enter value of y= 3

Enter value of z= 3

Product= 18

=== Code Execution Successful ===
```

main.cpp

Share | Run

```cpp
//Example 3
//Hierarchical Inheritance
#include <iostream>
using namespace std;

class A {
public:
    int x, y;
    void getdata() {
        cout << "\n Enter value of x and y:\n";
        cin >> x >> y;
    }
};

class B : public A {
public:
    void product() {
        cout << "\n Product= " << x * y;
    }
};

class C : public A {
public:
    void sum() {
        cout << "\n Sum= " << x + y;
    }
};

int main() {
    B obj1;
    C obj2;

    obj1.getdata();
    obj1.product();

    obj2.getdata();
    obj2.sum();

    return 0;
}
```

Output

```
 Enter value of x and y:
2 3

 Product= 6
 Enter value of x and y:
2 3

 Sum= 5

=== Code Execution Successful ===
```

Difference between multilevel and multiple inheritance

In multilevel inheritance, a class is derived from another derived class.

Structure:

Base Class → Derived Class → Further Derived Class

In multiple inheritance, a class is derived from more than one base class.

It combines features of multiple classes.

Structure:

Base Class 1 + Base Class 2 → Derived Class

Multilevel Inheritance Example:

main.cpp — Run

```cpp
//multilevel inheritance example
#include <iostream>
using namespace std;

class A {
public:
    void showA() {
        cout << "Class A" << endl;
    }
};

class B : public A {
public:
    void showB() {
        cout << "Class B" << endl;
    }
};

class C : public B {
public:
    void showC() {
        cout << "Class C" << endl;
    }
};

int main() {
    C obj;
    obj.showA();  // From class A
    obj.showB();  // From class B
    obj.showC();  // From class C
}
```

Output

```
Class A
Class B
Class C


=== Code Execution Successful ===
```

Multiple Inheritance Example:

main.cpp — Run

```cpp
// multiple inheritance example
#include <iostream>
using namespace std;

class A {
public:
    void showA() {
        cout << "Class A" << endl;
    }
};

class B {
public:
    void showB() {
        cout << "Class B" << endl;
    }
};

class C : public A, public B {
public:
    void showC() {
        cout << "Class C" << endl;
    }
};

int main() {
    C obj;
    obj.showA();  // From class A
    obj.showB();  // From class B
    obj.showC();  // From class C
}
```

Output

```
Class A
Class B
Class C


=== Code Execution Successful ===
```

EXERCISE

main.cpp

```cpp
//Exercise Q1
#include <iostream>
using namespace std;

class Base {
private:
    int privateInt;
protected:
    int protectedInt;
public:
    int publicInt;

    void setPrivateInt(int x) { privateInt = x; }
    void setProtectedInt(int x) { protectedInt = x; }
    void setPublicInt(int x) { publicInt = x; }

    int getPrivateInt() { return privateInt; }
    int getProtectedInt() { return protectedInt; }
    int getPublicInt() { return publicInt; }
};

class publicChild : public Base {
public:
    void showAccess() {
        cout << "Public Inheritance:\n";
        cout << "Accessing publicInt directly: " << publicInt << endl;
        cout << "Accessing protectedInt directly: " << protectedInt << endl;
        cout << "Accessing privateInt using getter: " << getPrivateInt() << endl;
    }
};

class protectedChild : protected Base {
public:
    void initialize() {
        setPrivateInt(10);
        setProtectedInt(20);
        setPublicInt(30);
    }

    void showAccess() {
```

Output

```
Public Inheritance:
Accessing publicInt directly: 3
Accessing protectedInt directly: 2
Accessing privateInt using getter: 1

Protected Inheritance:
Accessing protectedInt inside class: 20
Accessing publicInt inside class: 30
Accessing privateInt using getter: 10

Private Inheritance:
Accessing protectedInt inside class: 200
Accessing publicInt inside class: 300
Accessing privateInt using getter: 100


=== Code Execution Successful ===
```

main.cpp

```cpp
    }

    void showAccess() {
        cout << "\nProtected Inheritance:\n";
        cout << "Accessing protectedInt inside class: " << protectedInt << endl;
        cout << "Accessing publicInt inside class: " << publicInt << endl;
        cout << "Accessing privateInt using getter: " << getPrivateInt() << endl;
    }
};

class privateChild : private Base {
public:
    void initialize() {
        setPrivateInt(100);
        setProtectedInt(200);
        setPublicInt(300);
    }

    void showAccess() {
        cout << "\nPrivate Inheritance:\n";
        cout << "Accessing protectedInt inside class: " << protectedInt << endl;
        cout << "Accessing publicInt inside class: " << publicInt << endl;
        cout << "Accessing privateInt using getter: " << getPrivateInt() << endl;
    }
};

int main() {
    publicChild obj1;
    obj1.setPrivateInt(1);
    obj1.setProtectedInt(2);
    obj1.setPublicInt(3);
    obj1.showAccess();
    protectedChild obj2;
    obj2.initialize();
    obj2.showAccess();
    privateChild obj3;
    obj3.initialize();
    obj3.showAccess();

    return 0;
}
```

Output

```
Public Inheritance:
Accessing publicInt directly: 3
Accessing protectedInt directly: 2
Accessing privateInt using getter: 1

Protected Inheritance:
Accessing protectedInt inside class: 20
Accessing publicInt inside class: 30
Accessing privateInt using getter: 10

Private Inheritance:
Accessing protectedInt inside class: 200
Accessing publicInt inside class: 300
Accessing privateInt using getter: 100


=== Code Execution Successful ===
```

main.cpp

Share | Run | Output

```cpp
//Exercise Q2
#include <iostream>
using namespace std;

class Teacher {
    string Name;
    int Age;
    string Institute;
public:
    void setTeacher(string n, int a, string i) {
        Name = n;
        Age = a;
        Institute = i;
    }
    string getName() { return Name; }
    int getAge() { return Age; }
    string getInstitute() { return Institute; }
};
class HumanitiesTeacher : public Teacher {
    string Department = "Humanities";
    string CourseName,  Designation;
public:
    void setDetails(string c, string d) {
        CourseName = c;
        Designation = d;
    }
    void display() {
        cout << "\nHumanities Teacher:\n";
        cout << "Name: " << getName() << endl;
        cout << "Age: " << getAge() << endl;
        cout << "Institute: " << getInstitute() << endl;
        cout << "Department: " << Department << endl;
        cout << "Course: " << CourseName << endl;
        cout << "Designation: " << Designation << endl;
    }
};
class ScienceTeacher : public Teacher {
    string Department = "Science";
    string CourseName, Designation;
public:
    void setDetails(string c, string d) {
        CourseName = c;
        Designation = d;
    }

    void display() {
```

```
Humanities Teacher:
Name: Ali
Age: 35
Institute: ABC University
Department: Humanities
Course: History
Designation: Lecturer

Science Teacher:
Name: Sara
Age: 40
Institute: XYZ University
Department: Science
Course: Physics
Designation: Professor

Maths Teacher:
Name: Ahmed
Age: 30
Institute: LMN College
Department: Maths
Course: Algebra
Designation: Assistant Professor


=== Code Execution Successful ===
```

main.cpp

Share | Run | Output

```cpp
    void display() {
        cout << "\nScience Teacher:\n";
        cout << "Name: " << getName() << endl;
        cout << "Age: " << getAge() << endl;
        cout << "Institute: " << getInstitute() << endl;
        cout << "Department: " << Department << endl;
        cout << "Course: " << CourseName << endl;
        cout << "Designation: " << Designation << endl;
    }
};
class MathsTeacher : public Teacher {
private:
    string Department = "Maths";
    string CourseName, Designation;

public:
    void setDetails(string c, string d) {
        CourseName = c;
        Designation = d;
    }

    void display() {
        cout << "\nMaths Teacher:\n";
        cout << "Name: " << getName() << endl;
        cout << "Age: " << getAge() << endl;
        cout << "Institute: " << getInstitute() << endl;
        cout << "Department: " << Department << endl;
        cout << "Course: " << CourseName << endl;
        cout << "Designation: " << Designation << endl;
    }
};
int main() {
    HumanitiesTeacher h;
    ScienceTeacher s;
    MathsTeacher m;
    h.setTeacher("Ali", 35, "ABC University");
    h.setDetails("History", "Lecturer");
    s.setTeacher("Sara", 40, "XYZ University");
    s.setDetails("Physics", "Professor");
    m.setTeacher("Ahmed", 30, "LMN College");
    m.setDetails("Algebra", "Assistant Professor");
    h.display();
    s.display();
    m.display();
    return 0;
}
```

```
Humanities Teacher:
Name: Ali
Age: 35
Institute: ABC University
Department: Humanities
Course: History
Designation: Lecturer

Science Teacher:
Name: Sara
Age: 40
Institute: XYZ University
Department: Science
Course: Physics
Designation: Professor

Maths Teacher:
Name: Ahmed
Age: 30
Institute: LMN College
Department: Maths
Course: Algebra
Designation: Assistant Professor


=== Code Execution Successful ===
```

main.cpp

Share Run

```cpp
//Exercise Q3
#include <iostream>
using namespace std;

class Weapons {
public:
    void WeaponsDescription() {
        cout << "Weapons are used for defense or attack.\n";
    }
};
class HotWeapons : public Weapons {
public:
    void HotWeaponsDescription() {
        cout << "Hot weapons use gunpowder or explode.\n";
    }
};
class Bombs : public HotWeapons {
public:
    void BombsDescription() {
        cout << "Bombs blow up.\n";
    }
};

class NuclearBombs : public Bombs {
public:
    void NuclearBombsDescription() {
        cout << "Nuclear bombs blow up, and use nuclear fission and fusion.\n";
    }
};

int main() {
    NuclearBombs n;

    n.WeaponsDescription();
    n.HotWeaponsDescription();
    n.BombsDescription();
    n.NuclearBombsDescription();

    return 0;
}
```

Output

```
Weapons are used for defense or attack.
Hot weapons use gunpowder or explode.
Bombs blow up.
Nuclear bombs blow up, and use nuclear fission and fusion.


=== Code Execution Successful ===
```

main.cpp

Share Run

```cpp
// Exercise Q4
#include <iostream>
using namespace std;

class Item {
protected:
    string name;
    int quantity;
public:
    void setItem(string n, int q) {
        name = n;
        quantity = q;
    }
};

class BakedGoods : public Item {
protected:
    float discount = 0.10;
};

class Cakes : public BakedGoods {
private:
    float price = 600;
public:
    float calculateBill() {
        float total = price * quantity;
        return total - (total * discount);
    }
};

class Bread : public BakedGoods {
private:
    float price = 200;
public:
    float calculateBill() {
        float total = price * quantity;
        return total - (total * discount);
    }
};

```

Output

```
Total Bill: 2170


=== Code Execution Successful ===
```

main.cpp

Share Run Output

```
    float price = 200;
public:
    float calculateBill() {
        float total = price * quantity;
        return total - (total * discount);
    }
};

class Drinks : public Item {
private:
    float price = 100;
    float discount = 0.05;
public:
    float calculateBill() {
        float total = price * quantity;
        return total - (total * discount);
    }
};

int main() {

    float total = 0;

        Cakes c;
        c.setItem("Cake", 3);
        total += c.calculateBill();

        Bread b;
        b.setItem("Bread", 2);
        total += b.calculateBill();

        Drinks d;
        d.setItem("Drink", 2);
    total += d.calculateBill();

    cout<< "Total Bill: " << total << endl;

    return 0;
}
```

```
Total Bill: 2170


=== Code Execution Successful ===
```